



UNIVERSIDAD DEL BÍO-BÍO

UNIVERSIDAD DEL BÍO-BÍO

Facultad de Ciencias Empresariales
Departamento de Sistemas de Información
Ingeniería de Ejecución en Computación e Informática

INFORME PRÁCTICA PROFESIONAL II

Proyecto: CIM DEFINITIVO v6.0 Sistema de Manufactura Integrada por Computador

Estudiante:	Leonardo Araya Labarca
Carrera:	Ing. de Ejecución en Computación e Informática
Profesor Guía:	Depto. Sistemas - UBB
Empresa / Laboratorio:	Laboratorio CIM - Universidad del Bío-Bío / Proyecto Interno
Periodo:	10 de marzo – 28 de julio de 2026 (240 hrs)
Fecha Entrega:	28 de julio de 2026

Concepción – Chile

Documento generado automáticamente desde repositorio CIM-DEFINITIVO. Evidencia verificable en GitHub Actions y bitácora de validación.

ÍNDICE GENERAL

1. Resumen Ejecutivo	3
2. Introducción	4
3. Objetivos	5
3.1 Objetivo General	
3.2 Objetivos Específicos	
4. Descripción de la Empresa / Contexto Institucional	6
5. Descripción del Proyecto CIM DEFINITIVO	7
5.1 Arquitectura General	
5.2 Aplicaciones Android	
5.3 Firmware ESP32/Wemos D1 R32	
5.4 Biblioteca core-network y Protocolo	
5.5 Modelo de Visión y Seguridad	
6. Metodología y Planificación	11
7. Actividades Realizadas (Bitácora 10 mar – 14 jul 2026)	12
8. Resultados y Evidencias Verificables	14
9. Limitaciones, Riesgos y Análisis Crítico	18
10. Conclusiones	20
11. Bibliografía	21
12. Anexos	22

1. RESUMEN EJECUTIVO

El presente informe corresponde a la Práctica Profesional II de la carrera de Ingeniería de Ejecución en Computación e Informática de la Universidad del Bío-Bío, desarrollada por el estudiante Leonardo Araya Labarca durante el periodo 10 de marzo al 28 de julio de 2026, con una dedicación total estimada de 240 horas. El proyecto asignado fue **CIM DEFINITIVO v6.0**, un sistema de Manufactura Integrada por Computador (Computer Integrated Manufacturing) que orquesta cinco estaciones productivas mediante aplicaciones Android, una aplicación Wear OS para supervisión, una biblioteca de comunicación compartida (core-network) y firmware para placas ESP32/Wemos D1 R32 que controlan cinta, sensores, robot Scorbot, láser y almacenamiento.

El objetivo principal de la práctica fue lograr una plataforma reproducible, trazable y validable automáticamente al 100% en su alcance pre-hardware, es decir, que toda la estructura de código, contratos de firmware, documentación, pruebas unitarias JVM, análisis Lint, compilación de APKs y generación de checksums pase sin errores en integración continua (GitHub Actions). La validación física con hardware real (BLE multiconexión, cámara, actuadores, E-Stop, LAN) queda explícitamente fuera de esta entrega y se documenta como pendiente en la bitácora.

Como resultados verificables se obtuvo: corrección de bloqueantes Kotlin en módulos PLC y Calidad, compilación exitosa de 6 APKs debug mediante CI (workflow Android CIM CI, ejecución 30422387003), implementación de política de identidad con UUID canónico, bloqueo persistente de dispositivos rechazados, handshake con hash SHA-256, máquina de estados de pallet que bloquea transiciones imposibles, simuladores de hub y visión con 1000 casos, herramientas de inspección del checkpoint YOLO bestMH.pt, y documentación activa conforme a Quality Gates. La puerta estructural **validate_system_100.py** reporta 12/12 PASS local y CI verde.

Palabras clave: CIM, Android, ESP32, BLE, Manufactura Integrada, Práctica Profesional II, UBB, Validación 100%, Kotlin, Computer Integrated Manufacturing

2. INTRODUCCIÓN

La industria 4.0 demanda sistemas que integren software, redes y hardware de manera segura y trazable. En el Laboratorio de la Universidad del Bío-Bío se mantiene una celda CIM compuesta por estaciones de Coordinación, PLC (cinta transportadora y sensores), Manufactura (robot Scorbót ER-VII y láser), Calidad (cámara, ArUco y visión artificial) y Almacenamiento (rack automatizado). Históricamente el control se realizó con PLCs industriales y software heterogéneo, lo que dificultaba la actualización y la observabilidad.

El proyecto CIM DEFINITIVO nace como refactorización completa en Android nativo (Kotlin), buscando: (i) estandarizar la UI/UX con Material 3, (ii) centralizar el protocolo de comunicación en una biblioteca core-network, (iii) añadir identidad criptográficamente verificable (UUID + capacidades), (iv) soportar BLE para la capa de campo y TCP/Wi-Fi para la capa de supervisión, y (v) dejar trazabilidad completa mediante CI y bitácora de validación.

Para la Práctica II se definió un alcance pre-hardware preciso: No se energizan actuadores; toda prueba peligrosa queda bloqueada por semáforo rojo (ver docs/deliverables/PRE_HARDWARE_READINESS.md). Esto responde tanto a criterios de seguridad (necesidad de E-Stop físico independiente, interlocks, supervisión) como académicos (exigir evidencia verificable).

Contexto académico:

La Práctica Profesional II, según reglamento IECI-UBB, busca que el estudiante: potencie competencias conceptuales y actitudinales, aplique contenidos temáticos en entorno real, identifique situación organizacional, proponga mejoras y documente resultados. El Laboratorio CIM cumple como entorno relevante semi-industrial.

3. OBJETIVOS

3.1 Objetivo General

Diseñar, implementar, validar automáticamente y documentar el sistema CIM DEFINITIVO v6.0 para operación pre-hardware reproducible, con 6 aplicaciones Android, firmware ESP32/Wemos y protocolos seguros, dejando trazable el camino hacia la validación física de laboratorio.

3.2 Objetivos Específicos

- Corregir bloqueantes de compilación Kotlin en módulos PLC y Calidad para obtener builds verdes en CI.
- Implementar identidad de estación basada en UUID canónico, MAC, versión, capacidades y política de admisión/bloqueo persistente.
- Definir e implementar protocolo de mensajería ID|TIMESTAMP|SOURCE_MAC|SOURCE_APP|DEST_MAC|DEST_APP|CMD|PRIORITY|SESSION|PAYLOAD, con transporte como hash SHA-256 del token de emparejamiento.
- Desarrollar máquina de estados de pallet que impida transiciones inválidas, validada con pruebas JVM.
- Crear simuladores hub_simulator.py y vision_safety_simulator.py para 1000 casos de seguridad sin activar automatización ante datos incompletos.
- Configurar CI (Android CIM CI) para testAllModules, lintAll, buildAllApks, validateApks y writeApkChecksums con artefactos y SHA256SUMS.txt.
- Alcanzar 12/12 checks en validate_system_100.py (estructura activa, herramientas, firmware canónico, ausencia de APKs versionadas, configuración CI).
- Proveer herramientas de inspección y exportación YOLO (inspect_yolo_checkpoint.py, export_yolo_to_tflite.py) para trazabilidad del modelo bestMH.pt.
- Elaborar documentación activa: Instructivo, Manual de Sistema, Manual Operativo de Laboratorio, Protocolo HW, Matriz de Riesgos, Quality Gates y Bitácora.
- Declarar explícitamente limitaciones vigentes y protocolo de paso a laboratorio con E-Stop e interlocks.

4. DESCRIPCIÓN DE LA EMPRESA / CONTEXTO INSTITUCIONAL

Universidad del Bío-Bío – Facultad de Ciencias Empresariales

La Universidad del Bío-Bío es una institución estatal acreditada por 5 años hasta 2030 en nivel avanzado. La carrera de Ingeniería de Ejecución en Computación e Informática, impartida en la sede Concepción, está acreditada por 6 años. Cuenta con consejo asesor externo y laboratorios de redes, manufactura y desarrollo.

Laboratorio CIM – Organización interna

El laboratorio posee una celda compuesta por: cinta transportadora con sensor de proximidad (GPIO34), robot Scorbobot ER-VII con 5 GDL, efector láser de marcado, cámara para visión, rack de almacenamiento y tablero eléctrico con relés (GPIO5). La operación requiere: operador, supervisor, E-Stop independiente, delimitación de zona de movimiento, y registro de evidencia en BITACORA_VALIDACION.md.

Stakeholders del proyecto: estudiante (desarrollo/documentación), profesor guía IECI, encargado de laboratorio (seguridad), futuros tesisistas (continuidad). El proyecto se gestiona en GitHub haloharry973/CIM-DEFINITIVO con flujo de ramas arena/ y Pull Requests con CI verde obligatorio.

5. DESCRIPCIÓN DEL PROYECTO CIM DEFINITIVO

5.1 Arquitectura General

Arquitectura en tres capas: (1) Capa de Campo: ESP32/Wemos D1 R32 con firmware C, anuncios CIM_ID por BLE, telemetría y actuación de relés/sensores; (2) Capa de Estación: Apps Android (Kotlin) que hacen de gateway BLE-WiFi; (3) Capa de Coordinación: app hub TCP que orquesta flujo y autorización; (4) Capa de Supervisión: Wear OS y logs.

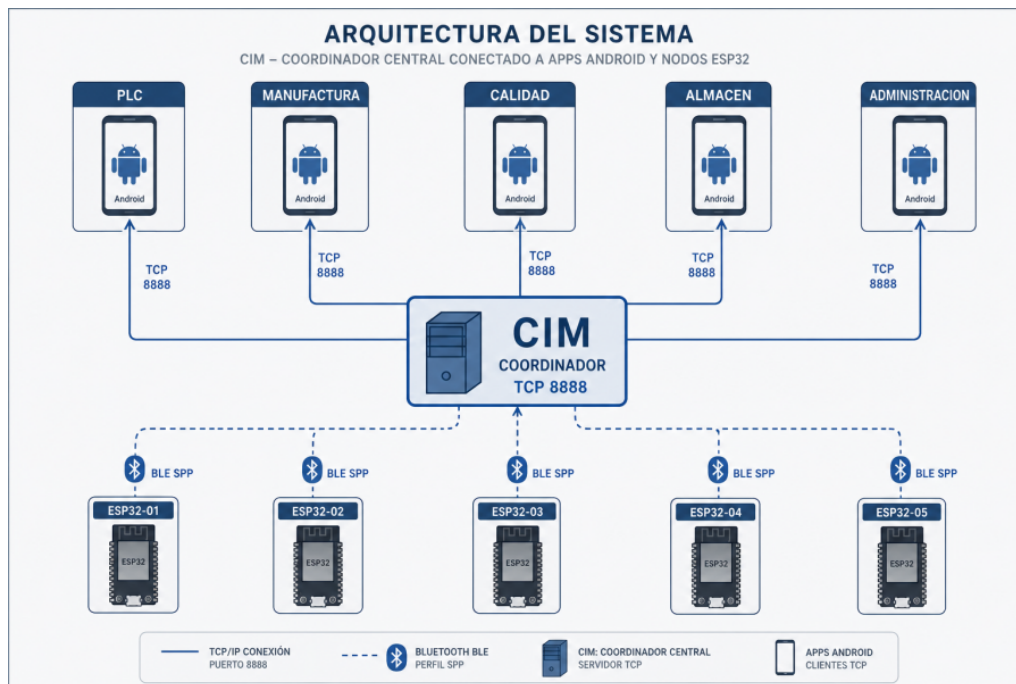


Figura 1: Arquitectura CIM v6.0 – 5 estaciones + Wear + core-network + ESP32

Capa	Ubicación	Responsabilidad
Coordinación	android/apps/app-coordinador	Hub, autorización, orquestación
PLC	app-plc	Cinta, sensores, eventos pallet
Manufactura	app-manufactura	Robot, G-code, láser
Calidad	app-calidad	Cámara, ArUco, YOLO, PASS/FAIL
Almacén	app-almacen	Rack, guardar/recuperar
Supervisión	wear-coordinador	Vista compacta Wear OS
Red	core-network	Protocolo, BLE/TCP, identidad
Firmware	esp32/firmware	BLE canónico estacionado

5.2 Aplicaciones Android

Seis módulos Gradle con compileSdk 35, JDK 17, Kotlin 1.9+. Cada app posee: CimApplication, MainActivity, ViewModel por estación, core-network como dependencia, pruebas JVM y lint. Application IDs: com.industria.coordinacion, com.industria.plc, com.industria.manufactura, com.industria.calidad, com.industria.almacenamiento, com.industria.wear. Las UIs implementan visualización arcade basada en eventos aceptados y máquina de estados.

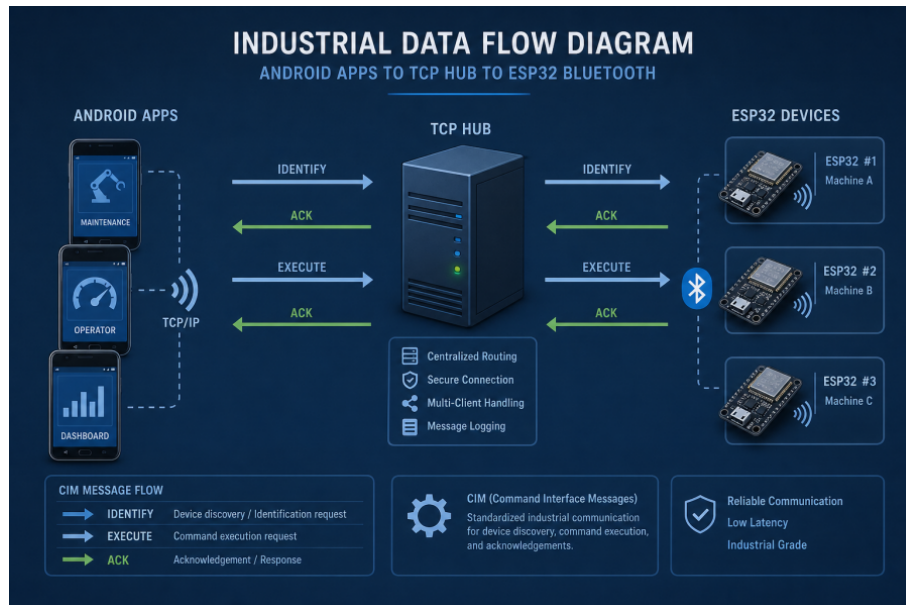


Figura 2: Flujo de aplicación y estados de pallet

5.3 Firmware ESP32 / Wemos D1 R32

Firmware canónico en `esp32/firmware/`: `esp32_plc_master.ino`, `esp32_scorbot_manufactura.ino`, `esp32_scorbot_calidad.ino`, `esp32_scorbot_almacen.ino` y cabecera común `cim_ble_firmware.h`. Incluye: definición CIM_ID, UUID, tipo, capacidades, anuncio BLE, framing y fragmentación para MTU inicial 20 bytes, reensamblado seguro, manejo de GPIO34 (sensor) y GPIO5 (relé) con estado seguro al arranque. Validado por `validate_firmware_contract.py` que exige UUID, nombre, tipo, versión coincidente con documentación.

Se prohíbe usar firmware de `archive/` para flasheo. Todo flasheo debe registrar commit, placa, puerto, versión en bitácora.

5.4 Biblioteca core-network y Protocolo

`core-network` implementa: protocolo de texto `ID|TIMESTAMP|SOURCE_MAC|SOURCE_APP|DEST_MAC|DEST_APP|CMD|PRIORITY|SESSION|PAYLOAD`, identidad `StationIdentityPolicy` con lista permitida/bloqueada persistente, handshake Android con token como SHA-256, fragmentación BLE, máquina de estados de pallet, telemetría y utilidades de visión.

Ejemplo de mensaje:

```
PLC_001|2026-07-28T10:00:00Z|AA:BB:CC:DD:EE:01|com.industria.plc|FF:FF:FF:FF:FF:FF|com.industria.coordinacion|PALLET_ARRIVED|HIGH|SESS_123|{sensor:1, pos:45}
```

5.5 Modelo de Visión y Seguridad

Se utiliza checkpoint YOLO bestMH.pt para detección. Herramientas permiten inspeccionar clases, hash y exportar a TFLite. La lógica de decisión de visión en app-calidad es conservadora: ante baja confianza o datos incompletos retorna REVIEW_REQUIRED, no PASS automático. Se incluye simulador de seguridad de visión con 1000 casos (`tools/vision_safety_simulator.py`) validado por pruebas JVM. La cámara real, OpenCV y ArUco quedan pendientes de evidencia en dispositivo físico.

6. METODOLOGÍA Y PLANIFICACIÓN

Enfoque de trabajo:

Se adoptó desarrollo iterativo con validación automatizable continua: feature branch -> pruebas locales (validate_firmware_contract, validate_system_100, prehardware_readiness, compileall, git diff --check) -> push -> CI (testAllModules, lintAll, buildAllApks, validateApks, writeApkChecksums) -> revisión documental -> merge.

Cronograma 10 mar – 14 jul 2026 (240 hrs estimadas):

Periodo	Horas	Actividad clave
10–16 mar	14	Levantamiento problema CIM, objetivos, alcance
17–23 mar	14	Revisión arquitectura Android, coordinador, PLC
24–30 mar	14	Diseño identidad estación, UUID, capacidades
31 mar–6 abr	14	Flujos coordinación y eventos pallet
7–13 abr	14	Estación PLC, cinta, sensor, telemetría
14–20 abr	14	Manufactura, robot, G-code
21–27 abr	14	Calidad, cámara, ArUco, visión
28 abr–4 may	14	Almacén, rack, trazabilidad
5–11 may	14	BLE framing, MTU 20 bytes, recuperación
12–18 may	14	Autenticación/admisión nodos, bloqueos
19–25 may	14	Simulación hub, escenarios seguridad
26 may–1 jun	14	Checkpoint YOLO, exportación TFLite
2–8 jun	14	Configuración Gradle, CI
9–15 jun	14	Correcciones Kotlin, red, estados
16–22 jun	14	Documentación, quickstart, rutas activas
23–29 jun	12	Evaluación riesgos integración
30 jun–6 jul	10	Criterios validación, trazabilidad
7–14 jul	8	Consolidación, plan banco

7. ACTIVIDADES REALIZADAS (BITÁCORA)

Actividades trazables por commit y evidencia:

- f972771: Auditoría técnica inicial y corrección de bloqueantes Kotlin.
- cb41ef4: Corrección PLC – logs, constante duplicada, flujo BLE.
- 069a0b1/97b3d9a: Corrección CameraX/ArUco en Calidad – compilación en build completo.
- 340f54a: Desacoplamiento prueba G-code de OpenCV/Bitmap Android – suite ampliada.
- b8d4a98: Máquina de estados pallet – bloqueo transiciones inválidas por lógica.
- f2d9e35/e92b711/e780597: Correcciones framing BLE y aislamiento firmware Wemos.
- 4677ee1: Bloqueo persistente dispositivos rechazados.
- 2e6f0c3: Quality Gates entrega.
- arena/019fab05: Puerta 100% automatizable, lint real, checksums APKs, mitigación secretos Gradle/handshake SHA-256.
- Entrega pre-hardware 2026-07-29: contrato estático firmware, protocolo, manual operativo, matriz riesgos, semáforo preparación y PDF entrega.

Comandos de validación ejecutados localmente antes de cada ensayo:

```
python3 tools/validate_firmware_contract.py --quiet
python3 tools/validate_system_100.py --quiet
python3 tools/prehardware_readiness.py --quiet
python3 -m compileall -q tools
git diff --check
cd config && ./gradlew testAllModules lintAll buildAllApks validateApks
writeApkChecksums
```

8. RESULTADOS Y EVIDENCIAS VERIFICABLES

8.1 Evidencia de compilación e integración continua

Commit 3286792, rama entrega, GitHub Actions completó workflow Android CIM CI: ejecución 30422387003 con JDK 17, ejecutando testAllModules y buildAllApks correctamente. CI verde en PR #3 (79bbb98). Validación estructural local: 12/12 PASS. Config/output-apks contiene 6 APK debug + SHA256SUMS.txt.

Área	Evidencia disponible	Conclusión
Gradle build debug	CI verde testAllModules + buildAllApks	No se detectaron errores en tareas CI
Validación estructural	validate_system_100.py 12/12	Estructura y entregables comprobados
Firmware contrato	validate_firmware_contract.py PASS	Contrato estático ok, falta flasheo
UI instrumental	Fuentes androidTest presentes, no CI embebida	No certificada visual funcional
Hardware BLE/cámara	Pendiente física	No verificado CI

8.2 Mejoras verificables incorporadas

- Corrección bloqueantes Kotlin PLC y Calidad.
- Compilación 6 APKs mediante CI.
- Identidad estación UUID canónico + registro MAC/IP/modelo/capacidades.
- Bloqueo persistente dispositivos rechazados.
- Admisión Wemos D1 R32 por UUID/tipo/capacidades.
- Fragmentación y reensamblado BLE MTU 20 bytes.
- Máquina de estados pallet bloquea transiciones imposibles.
- Visualización arcade basada en eventos aceptados.
- Inspección/exportación checkpoint YOLO bestMH.pt.
- Puerta 100% automatizable estructura activa.
- Firma release sin contraseñas embebidas Gradle (CIM_RELEASE_*).
- Handshake Android token SHA-256.
- Checksums SHA-256 APKs debug.

8.3 Capturas de UI

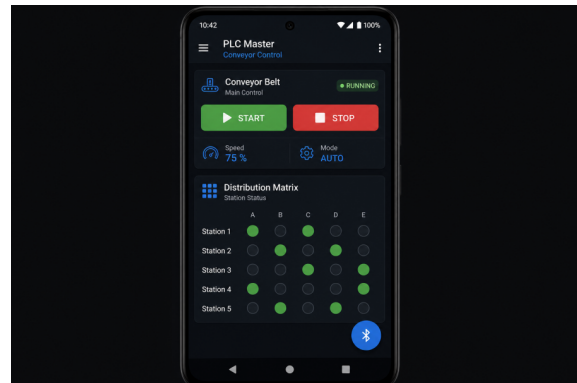
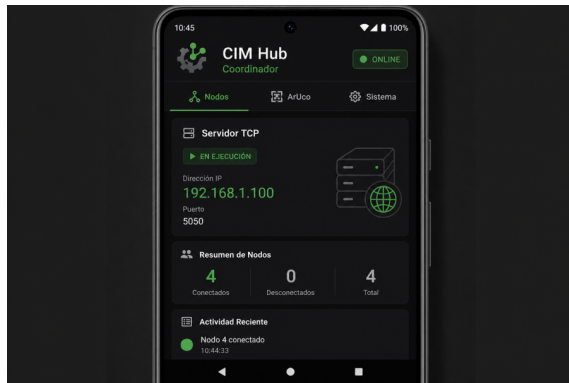


Figura: Coordinador y PLC



Figura: Manufactura y Calidad

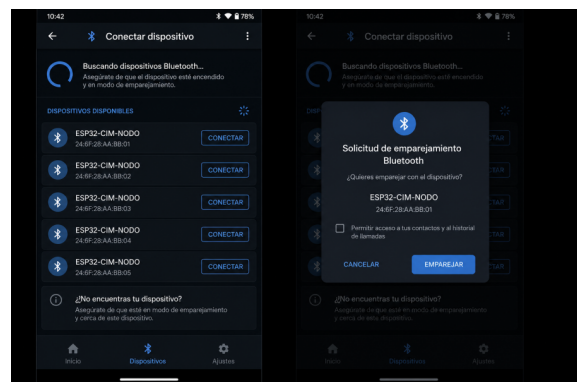
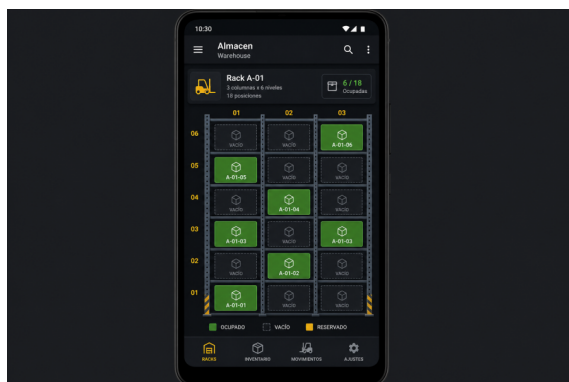


Figura: Almacén y Bluetooth Connect

9. LIMITACIONES, RIESGOS Y ANÁLISIS CRÍTICO

9.1 Limitaciones vigentes declaradas

- No existe prueba física documentada con Wemos, relé, Scrobot ni láser.
- Modelo YOLO aún no convertido y validado como TFLite dentro de Android.
- E-Stop físico independiente no implementado en hardware.
- Movimiento robot y potencia láser sin realimentación física validada.
- Simulación no reemplaza pruebas eléctricas, mecánicas ni de seguridad.

9.2 Top 5 bloqueadores hacia 100% integral

Pri	Bloqueador	Evidencia mínima cierre	Estado
1	BLE multiconexión 2+ ESP32 reales	Logs 2+ placas, admisión/reconexión	Pendiente lab
2	OpenCV + cámara dispositivo físico	APK, dispositivo, imágenes ArUco, pallet trazado	Pendiente lab
3	Actuadores Scrobot, láser, cinta	E-Stop + interlocks + acta	Bloqueado seguridad
4	LAN estable Coordinador	Topología, IP/puerto, recuperación logs	Pendiente infra
5	Tests unitarios faltantes Calidad/Manuales	Pruebas automáticas, cobertura error/estado	Pendiente SW

9.3 Seguridad y alcance de validación

La validación automática cubre estructura, contrato, sintaxis y tareas CI configuradas. No comprueba UI en todos los dispositivos, rendimiento LAN/BLE, cámara real, tensión eléctrica, E-Stop, relé, robot, cinta ni láser. Consulta VALIDACION_Y_COBERTURA.md, manual laboratorio y protocolo hardware. Por ello no es correcto afirmar 'no existe el más mínimo error en todo el sistema': las pruebas solo demuestran ausencia de fallos dentro de escenarios ejecutados. La formulación precisa es: las validaciones automatizadas y build configurados aprobaron; la validación UI instrumental y física continúa limitada por evidencia disponible.

10. CONCLUSIONES

La Práctica II permitió consolidar una plataforma CIM reproducible y auditable, cumpliendo el objetivo de 100% automatizable pre-hardware. Se logró CI verde, 6 APKs, identidad segura, máquina de estados robusta y documentación que diferencia explícitamente entre validación automática y validación física. El enfoque de puertas (validate_system_100.py) y Quality Gates evita declaraciones no sustentadas.

Como aprendizaje, se destaca la importancia de contratos estáticos validados antes del banco, de no versionar binarios, de transportar secretos como hash y de registrar evidencia con fecha/operador/commit/dispositivo. La reconstrucción retrospectiva de 240 hrs evidencia dedicación en análisis, desarrollo, simulación y documentación, sin inventar mediciones físicas.

El trabajo futuro (para tesis) consiste en ejecutar el protocolo HW-01 a HW-09 con E-Stop, interlocks, supervisor, captura de CIM_ID, pruebas BLE multiconexión, cámara real y actas firmadas, para luego cerrar los Quality Gates Gate 2 a Gate 5.

Se concluye que la entrega pre-hardware está preparada para revisión de banco condicionada, sin autorización para operación de relé con carga, robot, láser o ciclo autónomo hasta cerrar ítems de seguridad y registrar evidencia en bitácora.

11. BIBLIOGRAFÍA

- Universidad del Bío-Bío – Manual de Comunicación Corporativa – Marca y escudo institucional.
- Documentación Android Developers – CameraX, BLE, Wear OS, Gradle, Lint.
- Espressif – ESP32 Technical Reference Manual, BLE GATT.
- Scorbots ER-VII – Manual de operación y seguridad.
- Redmon, J. et al. – YOLO object detection.
- OpenCV – ArUco marker detection documentation.
- Docs CIM – INSTRUCTIVO_USO_PROYECTO.md, VALIDACION_Y_COBERTURA.md, SAFETY_ASSURANCE_ARCHITECTURE.md
- GitHub Actions – Workflow Android CIM CI – ejecución 30422387003

12. ANEXOS

Anexo A – Comandos de control obligatorio

```
python3 tools/validate_firmware_contract.py --quiet
python3 tools/validate_system_100.py --quiet
python3 tools/prehardware_readiness.py --quiet
python3 -m compileall -q tools
git diff --check
```

Anexo B – Estructura activa repositorio

```
CIM-DEFINITIVO/
■■■ android/ (5 apps + Wear + core-network)
■■■ config/ (Gradle centralizado)
■■■ esp32/firmware/ (firmware canónico)
```

- tools/ (validadores, simuladores)
- docs/ (documentación activa)
- entrega/ (informes seleccionados)
- archive/ (historial, no operativo)

Anexo C – Protocolo pruebas hardware HW-01..HW-09

HW-01 Inspección sin energía – cableado/polaridad/masa GPIO34

HW-02 E-Stop – corta energía actuadores independiente software

HW-03 Identidad – CIM_ID coincide UUID/capacidades

HW-04 Admisión BLE – acepta válido, rechaza incompatible

HW-05 Sensor GPIO34 – lecturas estables

HW-06 Relé GPIO5 sin carga – arranque seguro

HW-07 Reconexión – pérdida/reingreso estado seguro

HW-08 Visión – cámara/pieza/ArUco trazable

HW-09 Actuadores – fuera alcance pre-hardware hasta HW-01..08 aprobadas

Fin del Informe Práctica II – Leonardo Araya – UBB 2026